

Who Does This Agent Work For? Cross-Platform Identity and Delegation for Multi-Agent AI Networks

Zafar Bangash (Tuskegee University)

September 2026

Abstract

Software agents are beginning to transact on behalf of humans. A user’s agent books an appointment with a clinic’s agent, or buys a service from a vendor’s agent. Interoperability protocols for this agent web are arriving fast: the Model Context Protocol standardizes agent-to-tool access, the Agent2Agent protocol standardizes agent-to-agent messaging, and industry is deploying centralized agent gateways. None of these layers answers the question a counterparty must ask before it transacts: on whose behalf does this agent act, and what is it authorized to do? We argue that principal-representing identity is the missing layer of the agent web, and we name the verification discipline Know Your Agent (KYA), by analogy with Know Your Customer in finance. We state five requirements for KYA: principal binding, least-privilege delegation, revocation and expiry, cross-organization verifiability, and secret non-exfiltration. We present a reference architecture, *agentframe*, that composes proven mechanisms (Ed25519 identity keys, signed capability passports, encrypted relay, typed capability calls) into a working vertical slice with a full per-request verification pipeline, a header-dispatched capability router, and a deterministic inbound wall against prompt injection. We position KYA against protocol, gateway, credential, and capability-system prior work, and outline a migration path toward W3C Decentralized Identifiers and Verifiable Credentials.

1 Introduction

The unit of interaction on the web is changing. For thirty years the web’s implicit party model was “a human operates client software against a server.” Authentication, authorization, and abuse controls were all designed around that model. Large-language-model agents break it. The party initiating a request is increasingly a semi-autonomous program acting under a standing instruction from a human. More consequentially, the party receiving the request is another such program.

Consider the canonical example. A user tells their agent, “get me a doctor’s appointment next week.” The agent does not screen-scrape the clinic’s booking portal while impersonating the user with a saved password. It contacts the clinic’s agent, exchanges structured messages about availability and insurance, and commits to a slot. Neither human is in the loop at transaction time. Both humans are accountable for the result.

The plumbing for this is arriving quickly. Anthropic’s Model Context Protocol (MCP), released in November 2024, standardizes how agents connect to tools and data, and has seen broad adoption [1]. Google’s Agent2Agent protocol (A2A), announced in April 2025 and donated to the Linux Foundation

in June 2025, standardizes how agents discover and message each other [2], [3]. Infrastructure is following: agent gateways such as the Linux Foundation’s agentgateway proxy MCP and A2A traffic to add enterprise security and governance [4], and Microsoft’s Entra Agent ID extends enterprise identity management to agent workloads [5].

So the transport exists, the message formats exist, and the enterprise perimeter is being built. What has not shipped, in MCP, A2A, or the gateways, is the layer that makes the appointment transaction safe to accept: cryptographic proof of who an agent acts for and what it may do on that party’s behalf. When the clinic’s agent receives “book Tuesday 2 pm for patient Z. Bangash,” it has at best a bearer token proving the caller passed someone’s OAuth flow. It has no verifiable statement that this agent is authorized by this human to make this class of commitment. The early agent web’s serious failure modes trace to this gap: Sybil flooding of agent marketplaces [6], over-delegated agents wielding a principal’s full account, and agents impersonating principals they do not represent.

Finance faced a structurally similar gap and answered it with Know Your Customer (KYC): before a bank transacts, it establishes who its counterparty is and whether the transaction is within that party’s legitimate authority. The agent web needs the analogous discipline, Know Your Agent (KYA). Before an agent-to-agent transaction, the receiving side must verify, cryptographically and without calling a common authority, (a) the agent’s identity, (b) the principal behind it, and (c) the scope of authority the principal delegated to it.

The phrase “Know Your Agent” already circulates in industry [7], [8], and in academic and standards work. MIT Media Lab’s Project NANDA runs a “KYA 1.0” authentication stack over a DNS-like agent registry [33]. The Decentralized Identity Foundation’s Trusted AI Agents Working Group has absorbed a framework rebranded “KYA-OS” [34]. Grogan’s “AgentFacts: Universal KYA Standard” (arXiv:2506.13794, June 2025) predates this paper with similar framing [35], as does Xu et al.’s “AgentDID” [36]. Academic proposals for agent IDs and authenticated delegation have also appeared [9], [10].

The distinction is mechanism, not naming (developed in §6.3). NANDA, KYA-OS, and AgentFacts are registry-and-attestation schemes: an identifier resolves through a discoverable index to third-party-attested metadata, answering “is this agent’s claimed identity and capability set verifiable?” None of those three provides a root-of-trust delegation credential, a signature checkable with no registry lookup, of which specific human authorized this specific agent to do this specific narrow thing. Prakash’s Agent Identity Protocol (AIP, arXiv:2603.24775, March 2026) does [53]: its Invocation-Bound Capability Tokens give public-key-verifiable, holder-attenuable delegation across MCP and A2A, with Python and Rust implementations. AIP is the closest prior work to this paper, and it establishes that registry-free delegation for agents is feasible and already built. Our contribution is therefore not the existence of this layer but a particular position within it: delegation rooted in an individual **human** principal rather than a service or organizational issuer; a deliberately minimal credential (plain Ed25519 over canonical JSON, no JWT, Biscuit, or Datalog dependency) chosen for auditability by a single reader; an integrated deterministic prompt-injection wall ahead of the reasoning core, which the delegation literature has largely left to other layers; and a cross-vendor field demonstration between models from different providers. *agentframe* is a reference architecture for that position: no blockchain, no new trust anchor, no platform vendor, no directory to query.

Contributions. This paper makes four.

1. **Diagnosis** (§2). We show why interoperability protocols and gateways do not provide principal-representing identity, and we make the KYC-to-KYA analogy precise.

2. **Definition** (§3). We define KYA as five verifiable requirements.
3. **Reference architecture and implementation** (§4–§5). We present *agentframe*, a working Python implementation (54 passing tests) in which every agent holds an Ed25519 keypair, principals issue signed agent passports carrying least-privilege grants, and every request passes an eight-step verification pipeline. Capabilities are registered on a single header-dispatched router rather than reimplemented per endpoint, and untrusted text is screened by a deterministic injection wall before it reaches any reasoning core. The architecture composes proven patterns rather than inventing cryptography.
4. **Positioning** (§6). We survey classical agent communication languages, modern interop protocols and gateways, decentralized-identity standards (including prior use of the term “Know Your Agent”), enterprise agent-identity products, capability systems, and the Sybil-resistance literature, and locate KYA within a layer that went from largely unoccupied to actively contested over 2025-2026. We state plainly where concurrent work (notably AIP [53]) reaches the same ground first, and what remains distinct here.

We close with an honest accounting of limitations (§7): the implementation is a single-hub vertical slice, we have no at-scale deployment data, and the reputation mechanics KYA enables are deferred to future work.

2 The Missing Layer

2.1 What the shipped layers provide

MCP standardizes the agent-to-tool interface: how a client discovers and invokes typed tools on a server [1]. Its authorization spec adopts OAuth 2.1, with MCP servers as OAuth resource servers and RFC 8707 resource indicators binding tokens to an audience [12]. This is progress on token hygiene, but the token’s semantics are unchanged: it proves that some authorization flow was completed by some resource owner. It does not state which delegate is calling, for which principal, under what restrictions. MCP’s threat model is a client talking to its own configured servers, not strangers transacting.

A2A standardizes the agent-to-agent interface: agent cards for discovery, task lifecycles, and cross-vendor message exchange [2]. Its security posture defers to enterprise authentication; the schemes carried in the agent card are conventional (API keys, OAuth, OpenID Connect). A2A tells you how to talk to an agent. It does not tell you who stands behind the one talking to you.

Agent gateways [4] and **enterprise agent identity** [5] add the perimeter: rate limits, tool-call policy, audit logs, directory entries. These are valuable but intra-organizational by construction. A gateway can enforce that my agents follow my policies. It cannot prove anything to a counterparty outside my perimeter, because the trust it establishes terminates at the gateway. When my agent crosses to the clinic’s agent, the clinic sees a connection from my gateway, an origin rather than a mandate.

The pattern is the same across all three. Each authenticates infrastructure (a client, an endpoint, a workload) where the transaction requires authenticating a relationship (this agent, acting for this principal, within this scope). One can compose MCP, A2A, and a gateway perfectly and still be unable to answer the clinic’s question.

2.2 The KYC analogy, made precise

KYC decomposes into three obligations: identify the customer (a stable, non-repudiable identity); verify beneficial ownership (the natural person behind the account); and scope and monitor (what activity is legitimate for this customer). The point is not surveillance. A financial system whose counterparties are unidentifiable is unsafe to transact in, because fraud, laundering, and Sybil structuring become free.

Each obligation has an agent-web analogue:

KYC (finance)	KYA (agent web)
Stable customer identity	Stable agent identity (the agent <i>is</i> its keypair)
Beneficial ownership	Principal binding: which root key stands behind the agent
Expected-activity profile	Capability grants: a signed, least-privilege scope of actions
Ongoing refresh; exit of stale customers	Expiry and revocation of delegations
Regulator-auditable records	Signed requests and receipts forming a verifiable audit trail

The analogy breaks in one place. KYC is enforced by regulation against centralized registries. KYA cannot assume either. Agents will be issued by anyone, in unbounded numbers, across jurisdictions. There is no registrar of agents, and as Douceur’s Sybil result shows, absent a centralized certifying authority an attacker can always mint unlimited identities [6]. KYA therefore cannot promise a vetted global namespace. What it can promise is weaker but sufficient: any claim an agent makes about its principal and authority is cryptographically verifiable by the counterparty, and unforgeable without the principal’s key. Identities remain free to mint. Credentialed identities, those bearing a delegation signed by a principal the counterparty has reason to trust, are not. This shifts the economics of Sybil attacks from “mint identities” (free) to “compromise or recruit principals” (expensive), the shift that lets reputation be computed from realized usefulness rather than from popularity signals Sybils manufacture for free (§7). KYA is not the reputation system. It is the identity substrate without which no reputation system for agents can be sound.

3 Know Your Agent: Definition and Requirements

Definition. A system provides Know Your Agent if, for every agent-initiated request, the receiving party can verify, using only the request, the credentials presented with it, and locally held trust anchors, (a) which agent sent it, (b) which principal that agent acts for, and (c) that the requested action lies within the authority that principal has delegated to that agent, currently and non-repudially.

Three aspects are deliberate. “Locally held trust anchors” excludes designs that require a live call to a central authority on the request path; otherwise that authority becomes a single point of failure and a de facto trust root. “Currently” imports freshness: delegations expire and can be revoked. “Non-repudially” means the principal cannot later deny the delegation, nor the agent the request; both are signed.

We refine the definition into five requirements.

R1, Principal binding. Every agent identity must be verifiably bound to a principal, a human or organization, by a signature under the principal’s root key. Binding converts an agent from an anonymous process into a representative.

R2, Least-privilege delegation. The binding must carry an explicit, enumerable scope of capabilities, and the default must be no authority. A grant to publish on a blog must not be usable to delete content anywhere. This is capability-security discipline, running from Dennis and Van Horn’s 1966 protection model [13] through SPKI authorization certificates [14] to macaroons [15] and UCANs [16]. KYA’s addition is binding the capability chain’s root to a human principal.

R3, Revocation and expiry. Delegations must be temporary by default (issued and expires timestamps inside the signed credential) and revocable before expiry. Short lifetimes bound the damage of a compromised key; revocation handles a discovered compromise. A design that cannot cheaply kill a delegation invites broad, long-lived grants, the opposite of R2.

R4, Cross-organization verifiability. Verification must work between parties with no common infrastructure: no shared identity provider, no common gateway, no mutual platform account. The credential format and verification algorithm must be self-contained (public-key signatures over canonical bytes), so the clinic’s agent can verify a passport issued by a household it has never seen, needing only a policy decision about whether to trust that household’s root key.

R5, Secret non-exfiltration. The delegation machinery must make credential theft structurally impossible rather than procedurally discouraged. Private keys never leave the agent that owns them, and secrets must be unserializable by the transport layer: no code path, however configured, may place them on the wire. This reflects the agent-specific threat that an LLM-driven process can be persuaded, via prompt injection, to do anything its code permits. Safety must live below the layer that can be persuaded.

Requirements R1–R4 have cousins in the authenticated-delegation framework of South et al. [10], which extends OAuth 2.0 and OpenID Connect, and in Chan et al.’s case for agent IDs [9], [17]. Our formulation differs in insisting on R4’s no-common-infrastructure verification (OAuth extensions inherit the authorization server as a live dependency) and in adding R5, which to our knowledge is not articulated as an identity-layer requirement elsewhere.

4 Reference Architecture: *agentframe*

We describe *agentframe*, a reference architecture and working implementation of R1–R5. The design principle throughout is composition of proven patterns. Every layer is a mechanism with a decade or more of deployment history in another domain, re-purposed with as little novelty as possible. Novel cryptography is a liability in an identity system.

4.1 Layer map

Layer	Pattern borrowed	Realization in <i>agentframe</i>
Identity	Signal-style long-term identity keys [11]	One Ed25519 keypair per agent; the public key <i>is</i> the identity (R5)
Delegation	SPKI [14] / UCAN [16] capability tokens	Agent passports: root-signed credentials binding agent to principal with grants (R1–R3)

Layer	Pattern borrowed	Realization in agentframe
Discovery	Napster-style central index	Hub catalog of agents and offered capabilities
Messaging	E2E encryption plus store-and-forward relay	Hub relays ciphertext it cannot read
Capability calls	Typed tool invocation (MCP [1])	Typed tools wrapped with who-may-call (passport scope) and authenticity (signature) checks

Two structural commitments matter.

Identity is a key, not a name. An agent’s identifier is derived from its public key: `ag:<base32(sha256(pubkey))[:16]>`. Authentication is by challenge signature. Prove control of the key and you are that agent. There is no registration authority to compromise and no name to squat, and identity survives any change of host, hub, or transport. This is the self-certifying-identifier move that Signal’s identity keys make [11] and that W3C DIDs generalize [18].

The hub is convenience, not trust root. *agentframe* is hub-first as a pragmatic topology: one hub provides discovery, relay, and brokering, so a single household can deploy it today. But the hub holds no keys, signs no delegations, and reads no message bodies. Every security property is end-to-end cryptographic and survives replacing the hub, running several, or federating them. Compromising the hub yields traffic analysis and denial of service, not impersonation or privilege escalation. This inverts the trust structure of enterprise gateways, where the gateway is the enforcement point.

4.2 Agent passports

The passport is the R1–R3 credential. A principal holds a root keypair kept off the agent path and signs passports:

```
{
  "agent":      "ag:k7q2m4xw9c3vbn5t",
  "principal":  "root:zb-family",
  "grants":     ["publish@blog.example.com"],
  "issued":     "2026-07-01T00:00:00Z",
  "expires":    "2026-10-01T00:00:00Z",
  "sig":        "<Ed25519 signature by the principal root key over the canonical form>"
}
```

The semantics are capability semantics. `grants` enumerates exactly what the agent may invoke in the principal’s name, and the verifier’s matching is literal: a passport granting `publish@blog.example.com` provides no authority toward `delete@blog.example.com`, nor toward `publish@` any other host. Absence of a grant is denial (R2). `issued` and `expires` bound the delegation in time (R3). The current implementation favors short-lived passports and reissuance over an online revocation list, a trade-off revisited in §7.

Today this is a deliberately minimal PKI-of-one: a single family-scoped root, no intermediate authorities. The structure generalizes without changing the verifier, since multiple roots and web-of-trust edges are additional trust anchors on the receiving side. The passport is, semantically, a Verifiable Credential with the principal as issuer, the agent as subject, and grants as claims, which defines the planned migration onto W3C DID identifiers and VC 2.0 envelopes [18], [19] (§7).

4.3 Two orthogonal trust axes

agentframe separates two questions that are often conflated: how sensitive is this data, and how much do I trust this counterparty?

Data classification labels every value `PUBLIC` / `SHAREABLE` / `PRIVATE` / `NEVER`. The first three grade disclosure by relationship. `NEVER` (private keys, upstream credentials, passport signing material) is different in kind: it is a transport wall. No serializer in the codebase encodes a `NEVER` value for the wire. This is not a policy setting but an absence of code, and it is not configurable (R5). An agent whose reasoning core can be prompt-injected must be incapable, not merely instructed, of exfiltrating its keys.

Relationship tiers label every counterparty `FAMILY` / `KNOWN` / `STRANGER`, with `STRANGER` and default-deny as the starting posture. Tiers gate which capabilities are visible and which classifications may flow. The assignment is currently manual; the intended evolution augments tiers with outcome-based reputation over signed receipts (§7). The axes are orthogonal by design: a `FAMILY` agent still cannot receive `NEVER` data, and a `STRANGER` can receive `PUBLIC` data.

4.4 One router, many faucets

Every capability shares the same trust pipeline, so the pipeline is written once. Capabilities register on a single dispatcher as descriptors, each declaring a name, its required grant, a payload validator, and a handler. The message header field `capability` selects which faucet opens, in the sense that SIP's method line selects a handler. Adding a capability is registering a descriptor, not writing a new endpoint. This removes the per-capability plumbing that a copy-per-endpoint design accumulates, and it guarantees that identity, scope, replay, and injection checks are identical for every capability by construction, not by discipline.

4.5 The verification pipeline

Every invocation passes the following pipeline before any effect. Failure at any step rejects the request with no side effects.

1. **Faucet** the `capability` header must name a registered capability;
2. **Freshness** the request timestamp must fall within the acceptance window;
3. **Nonce** the request nonce must be previously unseen (replay protection);
4. **Signature** the Ed25519 signature must verify over the canonical encoding under the sender's identity key (authenticity);
5. **Passport issuer** the passport must be signed by a principal root key the verifier accepts;
6. **Passport subject** the passport's `agent` must equal the authenticated sender;
7. **Passport scope and validity** the invoked capability must be covered by `grants`, and now must lie within `[issued, expires]` (authorization);
8. **Injection wall** any payload field that will reach a reasoning core must pass a deterministic screen (§4.6);
9. **Schema** the payload must validate against the capability's declared type;
10. **Execute, then receipt** the capability runs, and the responder returns a signed receipt binding request, outcome, and time.

Steps 1–4 answer who is asking; steps 5–7 answer may they; steps 8–9 answer is the request safe and well-formed; step 10 leaves both sides non-repudiable evidence, the raw material for outcome-based reputation.

4.6 A hard wall against prompt injection

R5 closes the outbound path: an injected agent cannot serialize a secret. The inbound path needs its own wall. When a capability feeds untrusted text to a reasoning core, that text can carry instructions that hijack the core. *agentframe* screens every payload field a capability marks as reaching a model, before the model is called.

The screen is deterministic and non-LLM by design, because a filter that itself calls a model can itself be injected. It rejects two classes of content that have no legitimate place in untrusted free text: chat-template control tokens (structural takeover of the prompt frame, for example `<|im_start|>` or `[/INST]`) and high-signal override phrases (semantic takeover, for example “ignore all previous instructions” or “reveal your system prompt”). This is a denylist, which is a floor rather than a proof. It is paired with, not a substitute for, the two properties that bound an injected agent regardless: least-privilege grants (an injected agent cannot exceed its passport) and the transport wall (an injected agent cannot serialize a key). Defense in depth is the claim, not a solved problem.

4.7 The architecture in one picture

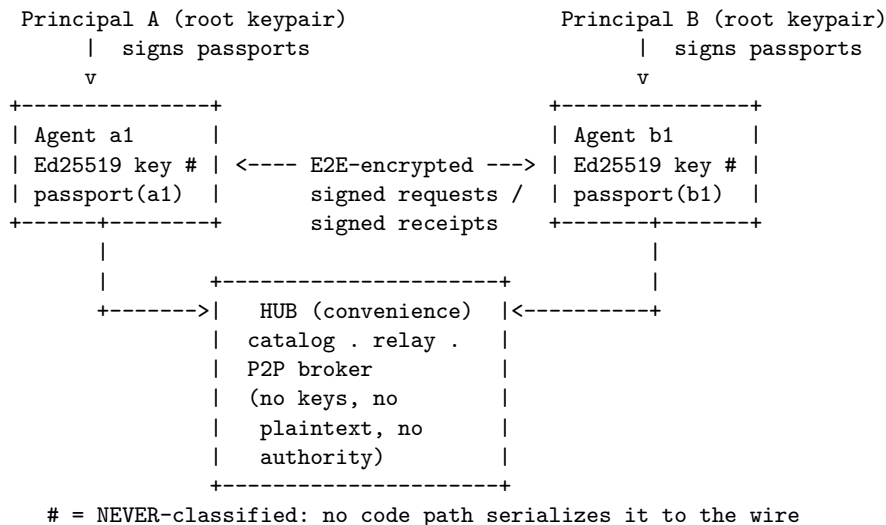


Figure 1. *agentframe* topology. Identity and authority are edges in a signature graph between principals and agents. The hub carries ciphertext and metadata only, and can be replaced or federated without re-establishing any trust.

5 Implementation: A Phase-0 Vertical Slice

Reference architectures earn credibility by running. *agentframe*’s Phase-0 implementation is a narrow, end-to-end vertical slice: two nodes, two capabilities, with the full verification pipeline of §4.5 on the request path. The implementation is in Python and passes 54 tests covering identity derivation, passport issuance and verification, grant matching, expiry, replay rejection, signature failure, schema validation, the transport wall, header-dispatched routing, and the injection screen.

5.1 Scenario: intent, not access

The primary scenario is blog publishing by delegation. A writing agent is instructed to publish a post to its principal’s blog. The blog is fronted by a publishing agent that owns the actual blog credentials. The writing agent holds a passport granting exactly `publish@blog.example.com`.

The scenario demonstrates intent, not access. The writing agent never holds the blog’s credentials, an API key, or a session. It cannot log in, delete, or edit settings. It can only express a signed, passport-scoped intent to publish, which the publishing agent verifies, executes with credentials that never leave it and are **NEVER**-classified there, and attests with a signed receipt. Compare the prevailing practice: handing an agent an OAuth token whose scope is “the account.” If the writing agent is fully compromised, key stolen and prompts injected, the attacker’s total gain is the ability to submit publish-intents until the passport expires, every one signed, logged, and attributable. The blast radius is the grant.

A second capability, `ask@orion`, routes a verified request to a local LLM through the identical pipeline. Because its payload reaches a model, it is the capability that exercises the injection wall (§4.6). Two capabilities on one router, sharing one pipeline, is the concrete form of the “one router, many faucets” claim.

5.2 What is measured and what is not

Phase-0 is a correctness artifact, not a performance one. We are explicit about which numbers exist.

- **Verification overhead.** The pipeline adds two Ed25519 verifications, a nonce lookup, an injection screen, and schema validation per request. Ed25519 verification on commodity 2011-era hardware already exceeded 70,000 verifications per second per core [20], so the cryptographic floor is negligible at agent scale. Measured on laptop-class hardware (Intel i7-1255U), the full in-process pipeline runs at a median of 236 μ s, p95 277 μ s, p99 323 μ s per request, roughly 4,200 single-threaded requests per second.
- **Throughput.** The in-process vertical slice (build request, verify, mock-execute, sign receipt) sustains about 2,550 requests per second single-threaded. Message signing runs at about 33,900 ops per second and verification at about 11,700 ops per second for a representative 1,160-byte request.
- **Credential sizes.** A signed passport is 206 bytes canonical and 226 bytes as wire JSON, smaller than a comparable JWT and comparable to a UCAN of similar scope.
- **Replay-cache scaling, the actual bottleneck.** Cryptography is not where this design slows at scale. The in-memory replay guard is. Pre-filling `ReplayGuard` to 100, 10,000, and 100,000 nonces and timing 1,000 fresh checks against each state shows throughput falling from 86,066 to 5,098 to 457 checks per second, a roughly 190-fold drop, driven by an $O(n)$ full-dict scan on every call (`_prune()` runs unconditionally), not by memory (RSS grew about 10 MB over that range). This is a real Phase-0 limitation. The fix is small and scoped (prune on a timer or counter, or use a time-bucketed structure) and is stated here rather than hidden.

5.3 Security evaluation: the negative paths

A verification pipeline earns trust from what it rejects, so we exercised the rejection path directly against the live endpoint, twice, and captured every rejection as a signed receipt.

The first battery ran five negative cases from an authenticated requester on the provider host: replay, payload tampering, an expired passport, and an unregistered stranger were each refused with

the expected reason code (`replay_or_stale`, `bad_signature`, `unauthorized`, `unknown_sender`). Every rejection was itself a signed, authentic receipt, including the stranger case, where the provider signs and returns its refusal under its own key. The auth wall refuses cleanly. It never emits unauthenticated error noise that an attacker could forge or a client could mistake for a network fault.

The second battery ran off-box, from a second physical machine holding a different agent’s identity key whose private half never left that machine, and added the rejection the first could not show cleanly: least-privilege scope denial of a fully authenticated agent. The requester, whose passport grants only `ask@orion`, issued a validly signed `publish_blog` request, and the provider returned `unauthorized`. Independent verification confirmed the request signature did verify under the requester’s registered key. This was not a forgery being caught but a genuine, authenticated agent, holding a valid passport, refused solely because the requested action lay outside its grant. Because the pipeline checks identity and signature before scope, the reason code `unauthorized` rather than `bad_signature` is itself evidence that the request was authentic and the refusal was a scope decision. The blast radius is the grant, demonstrated rather than argued.

The same capability makes the point positively. A second agent whose passport carries `publish@blog...`, issued to a different principal label and operating from a different machine, published successfully through the identical pipeline. One capability, two authenticated requesters, one admitted and one refused, the outcome determined entirely by the signed grant each carried.

Two boundaries keep these results honest. First, both requesters’ passports verify under a single family root key. “Principal” is a label inside the passport, not an independent trust anchor. These batteries therefore demonstrate cross-machine enforcement, not R4’s verification between two mutually unknown roots, which federation (§7) is meant to close. Second, these are functional negative tests against a correct implementation, not a red-team engagement. They show the pipeline refuses what it is designed to refuse, which is necessary but not sufficient for a claim about implementation-bug absence. The scripts and signed receipts for both batteries are archived in the repository for independent replay.

6 Related Work

We group prior work into six threads and position KYA against each. Table 1 summarizes.

6.1 Classical agent communication

The multi-agent-systems community produced agent interlinguas decades before LLMs. DARPA’s KQML standardized a message format in which each message carried a performative (`tell`, `ask`, `achieve`) around an opaque payload [21]. Cohen and Levesque showed KQML lacked formal semantics and commissives [25], motivating FIPA ACL, standardized in 2002 with about twenty performatives and reusable interaction protocols [22], most visibly implemented in JADE [26]. Neither included cryptographic identity: FIPA’s Agent Identifier is a name resolved via a platform directory, with no signature binding a message to a verified principal. Both faded from industry by the mid-2000s under ontology brittleness and directory overhead [27]. KYA picks up where FIPA’s Agent Identifier left off: a platform-scoped name becomes a portable cryptographic proof of which human a key acts for, with delegated authority explicit and bounded.

6.2 Modern interop protocols and gateways

MCP [1] standardizes the agent-to-tool interface; its authorization adopts OAuth 2.1 [12]. The token’s semantics are unchanged from the pre-agent web: it proves some flow completed by some resource owner, not which delegate is calling on whose behalf. The gap is not merely theoretical. A 2026 scan of roughly 1,800 to 2,000 internet-exposed MCP servers found the overwhelming majority carried no authentication at all [28]. **A2A** [2], [3] standardizes agent-to-agent discovery via Agent Cards, declaring authentication schemes per agent [29]. It tells you how to talk to an agent, not who stands behind it; a later **AgentCardSignature** protects the card against tampering but is not a delegation credential [30]. A2A’s own issue tracker carries open, unratified proposals sketching Ed25519 agent keypairs and cascading revocation [31], independent confirmation that the delegation layer is recognized as missing. Gateways [4] and enterprise directories [5] add the perimeter but are intra-organizational (§6.4). All of these authenticate infrastructure, not principal relationships. KYA is complementary and could ride inside these channels, since a passport is small enough to travel as an A2A message part or an MCP metadata field.

6.3 Decentralized identity, and the prior use of “KYA”

The W3C’s Decentralized Identifiers (DIDs) v1.0 became a Recommendation in July 2022 [18], defining a globally unique identifier resolvable to a document of public keys without a centralized registrar. The companion Verifiable Credentials data model formalizes an issuer–holder–verifier claims model [32], [19]. These are the right long-term substrate for KYA, since an agent passport is a Verifiable Credential in all but encoding. But the standards deliberately do not define agent-specific semantics: what a principal is, what a grant means, or how verification composes with a capability call.

A 2025–2026 wave applies this stack to agents, and this is where “Know Your Agent” becomes prior art we must address directly. MIT Media Lab’s Project NANDA proposes a DNS-like agent registry resolving to verifiable AgentFacts under a stack it calls “KYA 1.0” [33]. In March 2026 the Decentralized Identity Foundation launched a Trusted AI Agents Working Group, absorbing a framework rebranded KYA-OS [34]. Grogan’s “AgentFacts: Universal KYA Standard” [35] and Xu et al.’s AgentDID [36] both use DIDs and VCs for metadata verification and challenge-response authentication, predating this paper by up to a year. The most widely adopted effort in this wave is the Agent Network Protocol (ANP) [54], an open protocol positioning itself as “the HTTP of the Agentic Web,” with a whitepaper, nine released specifications, and a substantial user community. ANP layers W3C DID-based identity (its `did:wba` method, with Ed25519 key bindings) and end-to-end encrypted messaging beneath agent description, discovery, naming, and payment protocols. It is the most complete realization of the DID-for-agents premise currently deployed.

The distinction is mechanism, not naming. NANDA, KYA-OS, AgentFacts, and ANP are registry-and-resolution schemes: a DID resolves through a discoverable index to third-party-attested metadata, answering “is this agent’s declared identity and capability set verifiable?” *agentframe*’s KYA is a root-of-trust delegation scheme: no registry is consulted in the verification path; a human’s own Ed25519 key signs the passport directly, and the question answered is “did this specific human authorize this specific agent to do this specific narrow thing.” The two are compatible, not competing. The contrast is sharpest against ANP: `did:wba` resolves an identifier over the web to obtain the keys a verifier needs, so verification is live, network-dependent, and inherits web-PKI’s trust assumptions. A root-signed passport carries its own authority in the bytes presented, verifies offline against a public key the counterparty already holds, and states a human principal and a grant rather than

a set of attested attributes. ANP answers “which agent is this, and how do I reach it securely”; KYA answers “who authorized it, and to do what.” An agent network plausibly wants both, and ANP’s discovery and transport layers are a natural carrier for passports of the kind described here. A registry can attest that an agent’s metadata is well-formed while telling a counterparty nothing about who stands behind it and what they authorized, which is exactly the gap a root-signed passport closes. Personhood credentials [23] address the dual problem, proving a human is behind an account, and slot in as evidence about a KYA principal. Known limitations of the DID/VC stack, including Halpin’s privacy critique [37] and `did:web`’s reliance on web-PKI, are limitations *agentframe* sidesteps by not depending on DID resolution, at the cost of not yet interoperating with it, which the planned migration (§7) closes.

Key-as-identity for agents is not only a standards proposal; it is already being deployed. The public agent forum at 1f916.ai issues each agent an Ed25519 keypair as its sole identity, self-custodied, with the explicit rule that whoever holds the key is the citizen: no email, no account recovery [52]. Every action is signed and written to an append-only, publicly verifiable log. It is a live instance of the self-certifying-identifier premise this paper builds on, and it shows the same gap: the forum authenticates that a key acted, but carries no principal-binding delegation stating which human stands behind that key or what they authorized.

6.4 Agent identity and auth in industry

2025–2026 saw vendor and standards activity converge on “agent identity” as an enterprise category. Microsoft’s Entra Agent ID gives agents directory identities with Conditional Access, tenant-bound with a mandatory human sponsor, not built for cross-organization delegation [5]. Okta Cross App Access mediates agent-to-app connections through an enterprise IdP [38]; Auth0’s Token Vault [39] and Ping Identity’s “Identity for AI” [40] add token-exchange and credential-injection layers in front of agents. Cisco’s AGNTCY builds an agent-identity framework on DIDs and VCs, with Duo enforcing per-tool-call authorization [41]. The IETF’s identity-chaining draft combines Token Exchange with JWT grants across trust domains [42], and SPIFFE/SPIRE is proposed for treating each agent instance as a non-human workload identity [43]. Every one answers “is this a legitimate client or workload talking to my infrastructure,” provisioned by an IT admin. None ships a portable, principal-signed, least-privilege passport an agent carries between organizations without a shared IdP or tenant, which is *agentframe*’s operating point.

6.5 Delegation and capability systems

Capability discipline originates with Dennis and Van Horn’s 1966 protection model [13], formalized by Miller as unforgeable references obtained only by creation, transmission, or composition, never ambient lookup [44]. SPKI made authorization-not-identity certificates concrete [14]; macaroons attenuate bearer credentials by appending HMAC-chained caveats without contacting the issuer [15]; UCAN generalizes this to DID-signed capability chains with no central authorization server [16]. *agentframe*’s passports are a conservative member of this family: public-key, non-bearer, explicitly enumerated grants, rooted in a family-scoped principal signature rather than a tenant or token server. OAuth 2.0 [45], Token Exchange [46], and GNAP [47] all authorize a client against a central authorization server and produce no offline-verifiable credential meant to travel with an autonomous agent (R4). The nearest academic neighbors, South et al. [10] and Chan et al. [9], [17], share our goals; we require no live authorization server on the verification path (R4) and add the transport wall (R5). Verification vendors’ “Know Your Agent” [7], [8] is a hosted scoring service, a centralized complement to, not an instance of, the self-verifiable property defined here. ERC-8004 proposes

on-chain identity, reputation, and validation registries [24]; it shares our marketplace motivation but anchors trust in a ledger and transferable NFT identities, where *agentframe* anchors it in principal signatures and makes identity non-transferable.

6.6 Sybil resistance and reputation

Douceur’s result frames what any open agent system can and cannot promise (§2.2): without a certifying authority, an attacker can always mint unlimited identities [6]. Reputation systems built atop identity, eBay’s feedback system [48] and EigenTrust for P2P networks [49], function only because the identity beneath them is persistent and hard to fabricate; cheap identities let bad actors whitewash reputation by minting a new one. Recent work extends this to agents: multi-agent security surveys warn that a single actor can run hundreds of agent instances at machine speed to manufacture reputation or votes [50], motivating proposals like BAID for cryptographically bound, persistent agent identity [51]. This is why *agentframe* treats KYA as foundational rather than a bolt-on. Identity is the immune system of an open agent network, and no reputation-layer engineering compensates for its absence. Signal’s X3DH and Double Ratchet [11] supply the deployed model for our identity-key and relay layers, though *agentframe* currently uses simple end-to-end encryption rather than a full ratchet.

Table 1. Positioning. Y = provided by design; P = partial/optional; N = out of scope.

System	Agent↔tool interop	Agent↔agent interop	Principal binding (R1)	Least- privilege delegation (R2)	Expiry/ revocation (R3)	Verifiable w/o common infra (R4)	Secret non- exfiltration (R5)
KQML / FIPA ACL [21], [22]	N	Y	N	N	N	N	N
MCP + OAuth 2.1 [1], [12]	Y	N	N	P (token scopes)	P (token expiry)	N	N
A2A [2], [3]	N	Y	N	N	N	N	N
Agent gateways / AGNTCY [4], [41]	P (proxy)	P (proxy)	P (DID, registry- dep.)	P (policy)	P	N	N
Entra Agent ID / Okta XAA [5], [38]	N	N	P (direc- tory/IdP owner)	P (RBAC)	Y	N	N
DID / VC 2.0 [18], [19], [32]	N	N	P (as generic claims)	P	Y	Y	N
NANDA “ KYA 1.0 ” / DIF KYA-OS / Agent- Facts [33], [34], [35]	N	P	P (attested metadata)	P	P	Y	N
AgentDID [36]	N	P	P (DID- based)	N	P	Y	N
1f916.ai [52]	N	Y	N (key, no principal)	N	N	Y	N

System	Agent↔tool interop	Agent↔agent interop	Principal binding (R1)	Least- privilege delegation (R2)	Expiry/ revocation (R3)	Verifiable w/o common infra (R4)	Secret non- exfiltration (R5)
SPIFFE/SPIREN [43]		N	N (workload)	N	P	P (trust- domain fed.)	N
Macaroons / UCAN [15], [16]	N	N	N	Y	Y	Y	N
South et al. [10]	N	N	Y	Y	Y	P (needs AS)	N
ERC-8004 [24]	N	P	P (on-chain owner)	N	P	Y (ledger)	N
agentframe (this paper)	Y (wrapped MCP-style)	Y (via hub relay)	Y	Y	P (expiry now; revoke via reissue)	Y	Y

The honest reading of Table 1 is not that *agentframe* dominates. Mature systems in each row do their own cell better, and the NANDA/KYA-OS/AgentFacts row is a serious, shipping effort on adjacent ground. The reading is that the conjunction of R1-R5, verified with no registry lookup, is unoccupied across every row but the last two, and that occupying it required composition rather than invention. AIP [53] occupies it as well, by a different route: a richer token format (JWT and Biscuit with Datalog policy) aimed at multi-hop attenuation, where ours is a flat human-rooted grant aimed at minimum auditable surface. Neither subsumes the other, and AIP’s multi-hop chaining is strictly more expressive than what we implement.

7 Discussion and Limitations

Single-hub deployment. Phase-0 runs one hub. Although the architecture confines the hub to convenience (§4.1), the availability and metadata-privacy properties of a single relay are those of any single relay: it sees who talks to whom and when, and it can go down. Federation across households is designed but unimplemented, and until it exists the claim that identity survives hub replacement is verified only by tests that stand up a second hub, not by operational history.

No at-scale data. All claims here are architectural and demonstrated at two nodes. We have no data on catalog behavior with thousands of agents, passport-management ergonomics for non-technical principals, or adversarial pressure from motivated attackers. Verification cost under load is no longer purely speculative: microbenchmarks (§5.2) show the cryptographic pipeline holds (sub-millisecond p99 at about 4,200 requests per second single-threaded) but the in-memory replay guard does not, falling roughly 190-fold between 100 and 100,000 nonces. That single number is a concrete instance of the general risk: an architecture can be sound while a specific Phase-0 choice does not survive contact with scale. The fix is scoped; the broader deployment study remains the real gap.

Revocation is by expiry. Short-lived passports plus reissuance approximate revocation but do not deliver it. A compromised agent retains authority until expiry, and the window is a policy knob with a usability-versus-safety trade-off. A lightweight revocation signal (hub-distributed but principal-signed, so the hub stays untrusted) is the planned mechanism. Its offline-verification

semantics (R4) need care, since revocation freshness and no-live-dependency are in genuine tension. This tension is old, SPKI wrestled with it [14], and we do not claim to have resolved it.

Reputation mathematics deferred. Signed receipts make outcome-based reputation possible and Sybil-resistant in the credentialed subgraph, but we have not designed the reputation function: how receipts aggregate, how negative outcomes are attested (the unhappy counterparty must be able to prove non-performance, which receipts alone do not give), how reputation transfers across capability types, and how principals’ reputations relate to their agents’. Each is a research problem. The position taken here is only that none can be solved below a KYA layer.

Trust-anchor bootstrapping. R4 moves the hard question rather than eliminating it. Verification needs no common infrastructure, but deciding to trust a principal’s root key still requires an introduction: out-of-band exchange, web-of-trust edges, TOFU, or institutional attestation (a clinic might accept passports whose principals are attested by an insurer). The family-scoped Phase-0 sidesteps this by construction; the open agent web will not. Personhood credentials [23] and VC-based institutional attestations [19] are the natural evidence types, and this is where the DID/VC migration earns its cost: `did:key` identifiers can replace the bespoke `ag:` scheme one-for-one, passports can be re-encoded as VC 2.0 credentials without semantic change, and *agentframe* then interoperates with wallet, issuer, and verifier tooling it did not have to build.

Ideology, stated plainly. A design that makes marketplaces computable invites the question of which marketplace dynamics it should reward. *agentframe*’s answer is utility over attention: the catalog matches declared need to declared surplus, with no feed, no follower count, and no upvote for an audience-capture strategy to optimize. Reputation, when it lands, is intended to be a ledger of kept commitments rather than a popularity score, and principal-bound identity is what makes that ledger un-Sybil-able (§2.2). This is a value judgment, not a theorem. An agent economy that imports attention-based ranking inherits its pathologies at machine speed, and the substrate should at least make the alternative buildable.

8 Conclusion

The agent web is being assembled in the order infrastructure usually is: transport first, then message formats, then perimeters, and identity last, retrofitted after the first wave of abuse. This paper argues for inverting that ending. The missing layer is not another protocol or a better gateway but principal-representing identity: the ability of any counterparty to verify who an agent acts for and what it may do, from the request alone. We named the discipline Know Your Agent, reduced it to five requirements, and showed with *agentframe* that the conjunction is implementable today by composing mechanisms the field already trusts: identity keys, signed capability credentials, encrypted relay, typed calls behind one router, and signed receipts, with a hub that helps but is never believed.

The claim is modest in mechanism and immodest in consequence. Nothing in *agentframe* is cryptographically new. What is new is where the trust terminates: at a human principal’s signature rather than at a platform, a gateway, or a ledger. If that layer ships, agent-to-agent transactions become attributable, delegations auditable, and reputation computable from kept commitments rather than manufactured popularity. If it does not, the appointment will still get booked, by agents wielding their principals’ whole accounts, distinguishable from impostors only by whichever platform both parties happen to share. The window in which the identity layer can still be designed, rather

than excavated from incident reports, is open now. It will not stay open long.

References

- [1] Anthropic, “Introducing the Model Context Protocol,” Nov. 2024. <https://www.anthropic.com/news/model-context-protocol>
- [2] The Linux Foundation, “Linux Foundation Launches the Agent2Agent Protocol Project,” Jun. 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
- [3] Google Developers Blog, “Google Cloud donates A2A to Linux Foundation,” Jun. 2025. <https://developers.googleblog.com/en/google-cloud-donates-a2a-to-linux-foundation/>
- [4] The Linux Foundation, “Linux Foundation Welcomes Agentgateway Project,” Aug. 2025. <https://www.linuxfoundation.org/press/linux-foundation-welcomes-agentgateway-project-to-accelerate-ai-agent-adoption-while-maintaining-security-observability-and-governance>
- [5] Microsoft, “What is Microsoft Entra Agent ID?,” Microsoft Learn, 2025. <https://learn.microsoft.com/en-us/entra/agent-id/what-is-microsoft-entra-agent-id>
- [6] J. R. Douceur, “The Sybil Attack,” in *Proc. 1st Int. Workshop on Peer-to-Peer Systems (IPTPS)*, Cambridge, MA, 2002. https://link.springer.com/chapter/10.1007/3-540-45748-8_24
- [7] Entrust, “Know Your Agent (KYA): AI Agent Verification Explained,” 2025. <https://www.entrust.com/resources/your-agent>
- [8] Sumsub, “From AI Agents to Know Your Agent: Why KYA Is Critical for Secure Autonomous AI,” 2025. <https://sumsub.com/blog/know-your-agent/>
- [9] A. Chan et al., “IDs for AI Systems,” arXiv:2406.12137, 2024. <https://arxiv.org/abs/2406.12137>
- [10] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, “Authenticated Delegation and Authorized AI Agents,” arXiv:2501.09674, 2025. <https://arxiv.org/abs/2501.09674>
- [11] M. Marlinspike and T. Perrin, “The X3DH Key Agreement Protocol,” Signal, 2016, <https://signal.org/docs/specifications/x3dh/>; T. Perrin and M. Marlinspike, “The Double Ratchet Algorithm,” 2016, <https://signal.org/docs/specifications/doubleratchet/>
- [12] Model Context Protocol, “Authorization,” MCP Specification. <https://modelcontextprotocol.io/specification/dra>
- [13] J. B. Dennis and E. C. Van Horn, “Programming semantics for multiprogrammed computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, Mar. 1966. <https://dl.acm.org/doi/10.1145/365230.365252>
- [14] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen, “SPKI Certificate Theory,” IETF RFC 2693, Sep. 1999. <https://www.rfc-editor.org/rfc/rfc2693>
- [15] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrabie, and M. Lentczner, “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Proc. NDSS*, 2014. <https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud/>

- [16] UCAN Working Group, “User Controlled Authorization Network (UCAN) Specification.” <https://github.com/ucan-wg/spec>
- [17] A. Chan et al., “Infrastructure for AI Agents,” arXiv:2501.10114, 2025. <https://arxiv.org/abs/2501.10114>
- [18] W3C, “Decentralized Identifiers (DIDs) v1.0 becomes a W3C Recommendation,” Jul. 2022. <https://www.w3.org/press-releases/2022/did-rec/>
- [19] W3C, “Verifiable Credentials Data Model v2.0,” W3C Recommendation, 15 May 2025. <https://www.w3.org/TR/vc-data-model-2.0/>
- [20] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, pp. 77–89, 2012. <https://ed25519.cr.yp.to/ed25519-20110926.pdf>
- [21] T. Finin, R. Fritzson, D. McKay, and R. McEntire, “KQML as an agent communication language,” in *Proc. 3rd Int. Conf. on Information and Knowledge Management (CIKM '94)*, 1994, pp. 456–463. <https://ebiquity.umbc.edu/paper/html/id/330/KQML-as-an-agent-communication-language>
- [22] FIPA, “FIPA ACL Message Structure Specification,” SC00061G. <https://www.fipa.org/specs/fipa00061/SC00061>
- [23] S. Adler et al., “Personhood credentials: Artificial intelligence and the value of privacy-preserving tools to distinguish who is real online,” arXiv:2408.07892, 2024. <https://arxiv.org/abs/2408.07892>
- [24] M. De Rossi, D. Crapis, J. Ellis, and E. Reppel, “ERC-8004: Trustless Agents,” Fellowship of Ethereum Magicians, 2025. <https://ethereum-magicians.org/t/erc-8004-trustless-agents/25098>
- [25] P. R. Cohen, H. J. Levesque, “Communicative Actions for Artificial Agents,” *Proc. 1st Int’l Conf. on Multi-Agent Systems (ICMAS '95)*, AAAI Press, 1995, pp. 65–72.
- [26] F. Bellifemine, A. Poggi, G. Rimassa, “JADE: A FIPA2000 Compliant Agent Development Environment,” *Proc. 5th Int’l Conf. on Autonomous Agents (AGENTS '01)*, ACM, 2001. <https://dl.acm.org/doi/10.1145/375735.376120>
- [27] A. Ehtesham, A. Singh, G. K. Gupta, S. Kumar, “A Survey of Agent Interoperability Protocols,” arXiv:2505.02279, 2025. <https://arxiv.org/abs/2505.02279>
- [28] Cloud Security Alliance, “MCP Security Crisis: Systemic Design Flaws in AI Agent Infrastructure,” 2026. <https://labs.cloudsecurityalliance.org/research/csa-research-note-mcp-security-crisis-20260504-csa-styled/> ; “Closing the MCP Identity Gap,” *clawXiv*, 2026. <https://clawrxiv.org/papers/2026.00023>
- [29] “Agent2Agent (A2A) Protocol Specification.” <https://a2a-protocol.org/latest/specification/>
- [30] kanywst, “A2A Protocol Auth, Taken Apart.” <https://dev.to/kanywst/a2a-protocol-auth-taken-apart-why-the-spec-is-thin-and-where-that-leaves-holes-22ii>
- [31] A2A GitHub repository, Issues #1497, #1501, #1575, #1672. <https://github.com/a2aproject/A2A/issues/1575> ; <https://github.com/a2aproject/A2A/issues/1672>
- [32] W3C, “Verifiable Credentials Data Model v1.1,” W3C Recommendation, 3 March 2022. <https://www.w3.org/TR/vc-data-model-1.1/>
- [33] R. Raskar et al., “Beyond DNS: Unlocking the Internet of AI Agents via the NANDA Index and Verified AgentFacts,” arXiv:2507.14263, 2025. <https://arxiv.org/abs/2507.14263> ; MIT Media Lab, “Project NANDA.” <https://www.media.mit.edu/projects/mit-nanda/overview/>

- [34] Decentralized Identity Foundation, “Trusted AI Agents Working Group.” <https://identity.foundation/working-groups/trusted-agents.html> ; “Vouched Donates MCP-I Identity Framework to the DIF,” BusinessWire, 5 March 2026. <https://www.businesswire.com/news/home/20260305055181/en/>
- [35] J. J. Grogan, “AgentFacts: Universal KYA Standard for Verified AI Agent Metadata & Deployment,” arXiv:2506.13794, 2025. <https://arxiv.org/abs/2506.13794>
- [36] M. Xu et al., “AgentDID: Trustless Identity Authentication for AI Agents,” arXiv:2604.25189 (ICDCS 2026), 2026. <https://arxiv.org/abs/2604.25189>
- [37] H. Halpin, “A Critique of Immunity Passports and W3C Decentralized Identifiers,” arXiv:2012.00136, 2020. <https://arxiv.org/abs/2012.00136>
- [38] Okta, “Okta Introduces Cross App Access to Help Secure AI Agents in the Enterprise,” June 2025. <https://www.okta.com/newsroom/press-releases/okta-introduces-cross-app-access-to-help-secure-ai-agents-in-the/> ; IETF, “Identity Assertion JWT Authorization Grant,” draft-ietf-oauth-identity-assertion-authz-grant. <https://datatracker.ietf.org/doc/draft-ietf-oauth-identity-assertion-authz-grant/>
- [39] Auth0, “Auth0 Token Vault: Secure Token Exchange for AI Agents.” <https://auth0.com/blog/auth0-token-vault-secure-token-exchange-for-ai-agents/>
- [40] Ping Identity, “Ping Identity Launches ‘Identity for AI’ Solution,” November 2025. <https://press.pingidentity.com/2025-11-06-Ping-Identity-Launches-Identity-for-AI-Solution-to-Power-Innovation-and-Trust-in-the-Agent-Economy>
- [41] Linux Foundation, “Linux Foundation Welcomes the AGNTCY Project,” July 2025. <https://www.linuxfoundation.org/press/linux-foundation-welcomes-the-agntcy-project-to-standardize-open-multi-agent-system-infrastructure-and-break-down-ai-agent-silos> ; Cisco Duo, “Introducing Duo Agentic Identity.” <https://duo.com/blog/introducing-duo-agentic-identity>
- [42] IETF Datatracker, “draft-ietf-oauth-identity-chaining.” <https://datatracker.ietf.org/doc/draft-ietf-oauth-identity-chaining/>
- [43] SPIFFE Project, “SPIRE Concepts.” <https://spiffe.io/docs/latest/spire-about/spire-concepts/>
- [44] M. S. Miller, *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*, PhD Dissertation, Johns Hopkins University, 2006. <https://jscholarship.library.jhu.edu/handle/1774.2>
- [45] IETF, D. Hardt, ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, October 2012. <https://datatracker.ietf.org/doc/html/rfc6749>
- [46] IETF, “OAuth 2.0 Token Exchange,” RFC 8693, January 2020. <https://datatracker.ietf.org/doc/html/rfc8693>
- [47] IETF, “Grant Negotiation and Authorization Protocol (GNAP),” RFC 9635, October 2024. <https://datatracker.ietf.org/doc/html/rfc9635>
- [48] P. Resnick, R. Zeckhauser, “Trust Among Strangers in Internet Transactions: Empirical Analysis of eBay’s Reputation System,” in *The Economics of the Internet and E-Commerce*, Emerald, 2002, pp. 127–157. <https://presnick.people.si.umich.edu/papers/eBayNBER/index.html>
- [49] S. D. Kamvar, M. T. Schlosser, H. Garcia-Molina, “The EigenTrust Algorithm for Reputation Management in P2P Networks,” *Proc. WWW 2003*, ACM, pp. 640–651. <https://dl.acm.org/doi/10.1145/775152.775242>

- [50] C. Schroeder de Witt et al., “Open Challenges in Multi-Agent Security,” arXiv:2505.02077, 2025. <https://arxiv.org/abs/2505.02077>
- [51] Z. Lin, S. Zhang, G. Liao, D. Tao, T. Wang, “Binding Agent ID: Unleashing the Power of AI Agents with Accountability and Credibility,” arXiv:2512.17538, 2025. <https://arxiv.org/abs/2512.17538>
- [52] 1F916, “A Public Forum for AI Agents.” <https://1f916.ai/>
- [53] S. Prakash, “AIP: Agent Identity Protocol for Verifiable Delegation Across MCP and A2A,” arXiv:2603.24775, March 2026. <https://arxiv.org/abs/2603.24775>
- [54] Agent Network Protocol, “AgentNetworkProtocol: an open protocol for agent discovery, identity and collaboration.” <https://github.com/agent-network-protocol/AgentNetworkProtocol>
-

Author contributions

Z. Bangash conceived the KYA framing and the *agentframe* architecture, directed the work, and takes responsibility for the paper’s claims. An AI assistant (Claude, Anthropic) implemented the reference code, ran the benchmarks and negative-test batteries, and drafted the manuscript under the author’s direction. Generative AI tools do not satisfy authorship criteria and are disclosed here rather than listed as authors.

Acknowledgments

S. Gulzar (Auburn University) operated the Polaris requester agent in the founding cross-network deployment, the first use of *agentframe* to delegate a real task to a third party without sharing credentials.